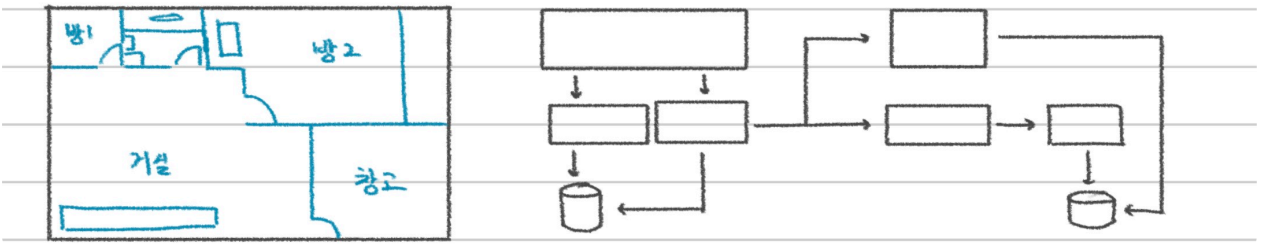
1. 소프트웨어 아키텍처 이해하기

소프트웨어 아키텍처를 이해하기 위해 집을 떠올려보자.

집의 구조가 아키텍처다. 예를 들어 집의 마당, 방의 개수, 층수, 크기 등이 여기에 해당한다. 예시에서 보듯이 구조적인 요소들은 나중에 바꾸기 어렵고, 그래서 중요한 부분이 된다. 이미 지은 집의 방 개수를 바꾸려면 벽을 허물어야 하는 것처럼 말이다.

집을 지으려면 먼저 선과 상자로 평면도를 그린다. 마찬가지로 소프트웨어 아키텍처 다이어그램은 소프트웨어의 구조를 정의한다.



평면도에는 바닥재의 종류, 벽의 색상, 침대의 방향 같은 세부사항이 정의되어 있지 않다. 이런 요소들은 구조적이지 않기 때문이다. 즉, 아키텍처가 아니라 **설계(디자인) 요소에 해당한다.**

1.1 소프트웨어 아키텍처의 차원들

> 책에서는 소프트웨어 아키텍처에 네 개의 차원이 있다고 설명한다.

첫 번째 차원 : 아키텍처 특성

- 아키텍처가 지원해야 하는 시스템의 측면을 기술한다.
- 예를 들어 확장성, 테스트 용이성, 가용성 등이 있다.

두 번째 차원 : 아키텍처 결정

- 시스템에 장기적이거나 상당한 영향을 미치는 중요한 결정을 포함한다.
- 예를 들어 데이터베이스의 종류, 시스템을 구성하는 서비스의 개수, 서비스 간 통신 방식 등이 있다.

세 번째 차원 : 논리적 컴포넌트

- 시스템 기능을 이루는 구성요소와, 그것들이 어떻게 상호작용하는지를 기술한다.
- 예를 들어 전자상거래 시스템에는 재고 관리, 결제 처리 같은 컴포넌트가 필요하다.

네 번째 차원 : 아키텍처 스타일

- 소프트웨어 시스템의 물리적인 구조와 형태를 정의한다.
- 예를 들어 모놀리식, 마이크로서비스 등이 있다.

차원	한 줄 요약	집으로 비유하면
아키텍처 특성	시스템이 **무엇을 잘해야** 하는가	단열이 잘 되는 집, 아이가 안전한 집
아키텍처 결정	구조를 제한하는 **규칙**	"거실을 거쳐야만 방에 갈 수 있다"
논리적 컴포넌트	시스템을 이루는 **기능 단위**	거실, 주방, 침실
아키텍처 스타일	시스템의 **전체 형태**	단독주택, 아파트, 타운하우스

소프트웨어 아키텍처에서의 컴포넌트

- 일반적으로 컴포넌트는 독립적으로 배포 가능하고 교체 가능한 소프트웨어 단위를 말한다.
- 캡슐화, 재사용성, 독립성을 띠며, 인터페이스를 기반으로 소통한다.
- 단, 세 번째 차원의 **논리적 컴포넌트**는 배포 단위가 아니라 **기능 단위**다. 하나의 애플리케이션 안에서 디렉토리나 네임스페이스로 구분될

수 있다. (1.4에서 자세히 다룬다)

이 네 가지 차원은 서로 맞물려야 한다.

아키텍처 스타일은 선택한 아키텍처 특성뿐만 아니라 아키텍처 결정과도 일치해야 한다. 논리적 컴포넌트 역시 특성, 스타일, 결정과 일치해야 한다. 예를 들어 높은 확장성이 핵심 특성인데 모든 기능을 하나의 배포 단위로 묶는 스타일을 고른다면, 차원끼리 서로 어긋나는 것이다.

1.2 첫 번째 차원 : 아키텍처 특성

> 아키텍처 특성은 소프트웨어 시스템 아키텍처의 기반을 형성한다.

이 특성이 없다면 아키텍처 관련 결정을 내리거나 중요한 트레이드오프를 분석할 수 없다. 예를 들어 새로운 시스템에 도입할 데이터베이스를 결정한다고 해보자. 관계형 데이터베이스, 단순한 key-value 데이터베이스, 그래프 데이터베이스 중 무엇을 골라야 할까?

정답은 어떤 아키텍처 특성이 중요한지에 따라 달라진다.

- 키로 값을 찾는 **단순 조회 성능**이 가장 중요하다면 key-value 데이터베이스가 유리하다.
- 데이터 사이의 **복잡한 관계를 탐색**해야 한다면(친구의 친구 추천 등) 그래프 데이터베이스가 유리하다.
- **데이터 무결성과 트랜잭션**을 지켜야 한다면 관계형 데이터베이스가 유리하다.

아키텍처 특성이라는 용어가 낯설게 느껴질 수 있지만, 사실 이미 익숙한 개념이다. `성능`, `확장성`, `신뢰성` 같은 요소들을 통칭해서 **비기능 요구사항**, **시스템 품질 속성**, 또는 영어로 단순히 **"-ilities"**(~성)라고 부르기도 한다.

- `신장성(extensibility)` : 시스템에 새로운 기능을 추가하기 쉬운 정도
- `민첩성(agility)` : 시스템을 빠르게 변경할 수 있는 능력
- `상호운용성(interoperability)` : 비즈니스 요청을 완료하기 위해 다른 시스템과 연결하고 상호작용할 수 있는 능력
- `내결함성(fault tolerance)` : 일부에서 치명적인 오류가 발생해도 시스템의 나머지 부분이 계속 동작할 수 있는 능력

> **확장성(scalability)과 신장성(extensibility)은 다르다.** 확장성은 사용자나 요청이 늘어날 때 버티는 능력이고, 신장성은 새로운 기능을 붙이기 쉬운 정도다.

모든 특성을 다 잘하는 아키텍처는 없다. 특성끼리 서로 충돌하는 경우가 많기 때문이다. 예를 들어 보안을 강화하면 성능이 떨어질 수 있다. 그래서 이 시스템에서 **가장 중요한 특성 몇 가지**를 골라내는 것이 아키텍처 설계의 출발점이다.

1.3 두 번째 차원 : 아키텍처 결정

> 아키텍처 결정은 시스템의 구조적 측면에 관한 선택으로, 장기적으로 중요한 영향을 미친다.

이 결정들은 **제약사항**이 되어 시스템을 기획하고 구축하는 방향을 잡아준다. 예를 들어 "사용자 인터페이스는 DB와 직접 통신하지 않고, 반드시 하위 서비스를 통해 데이터를 조회하고 갱신해야 한다"고 결정할 수 있다.

- 이 아키텍처 결정은 사용자 인터페이스 개발의 제약사항이 된다.

- 일반적으로 시스템이 더 크고 복잡할수록 더 많은 아키텍처 결정이 필요하다.

아키텍처 결정은 "무엇을 써라"보다 "무엇을 하면 안 된다"에 가까운 경우가 많다. 이렇게 제약을 두면 팀원 모두가 같은 구조를 지키게 되고, 시스템이 시간이 지나도 무너지지 않는다. 그래서 결정의 내용뿐만 아니라 **왜 그렇게 결정했는지**도 함께 기록해두는 것이 좋다.

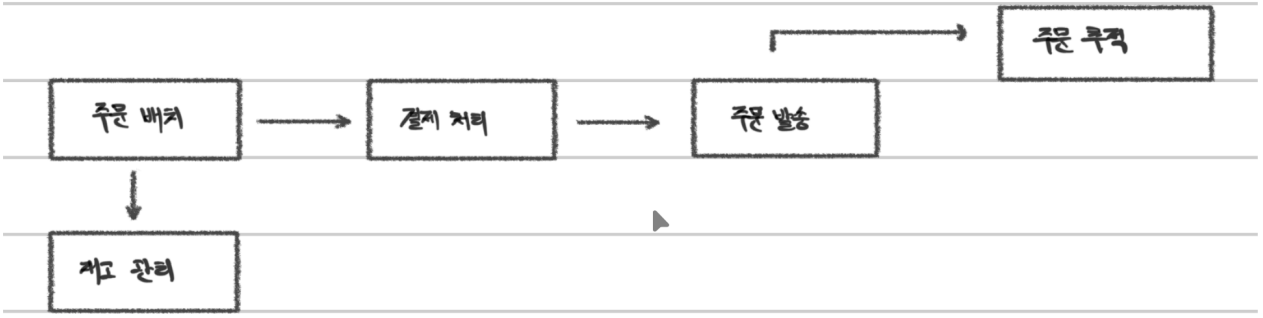
1.4 세 번째 차원 : 논리적 컴포넌트

> 논리적 컴포넌트는 시스템의 구성요소로, 집으로 비유하면 방에 해당한다.

논리적 컴포넌트는 주문 결제 처리, 재고 관리, 주문 추적 같은 기능을 수행한다. 시스템 안에서 논리적 컴포넌트는 보통 **디렉토리** 또는 **네임스페이스**로 표현된다.

```
app
└── order
    └── payment    ← "Payment Processing" 컴포넌트
```

디렉토리 `app/order/payment`와 네임스페이스 `app.order.payment`는 "Payment Processing"이라는 논리적 컴포넌트로 식별된다. 결제 처리 소스코드는 이 디렉토리에 저장되고 이 네임스페이스를 사용한다. 자바라면 패키지 `com.example.order.payment`가 이 역할을 한다.



논리적 컴포넌트는 시스템에서 항상 **잘 정의된 역할과 책임**을 가져야 한다. 즉, 무엇을 하는지에 대한 명확한 정의가 필요하다. 방마다 용도가 정해져 있어야 집이 정돈되는 것처럼, 컴포넌트의 책임이 모호하면 기능이 이곳저곳에 흩어지고 결국 구조가 무너진다.

1.5 네 번째 차원 : 아키텍처 스타일

> 아키텍처 스타일은 각기 다른 고유한 특성으로 소프트웨어 시스템의 전반적인 모습과 구조를 정의한다.

마이크로서비스 아키텍처 스타일

- 시스템을 독립적으로 배포되는 작은 서비스 여러 개로 나눈다.
- 확장성이 좋고, 빠르게 변화하는 환경에 대응할 수 있는 높은 수준의 민첩성을 제공한다.
- 대신 구조가 복잡하고 운영 비용이 많이 든다.

레이어드 아키텍처 스타일

- 코드를 프레젠테이션, 비즈니스, 영속성처럼 **기술적인 계층**으로 나눈다. 보통 하나의 배포 단위로 묶이는 모놀리식 형태다.
- 덜 복잡하고 비용이 적게 든다.
- 대신 시스템이 커질수록 확장성과 민첩성이 떨어진다.

이벤트 기반 아키텍처 스타일

- 서비스들이 직접 호출하는 대신 이벤트를 주고받으며 비동기로 동작한다.
- 높은 확장성을 제공하며, 매우 빠르고 응답성이 좋다.
- 대신 흐름을 추적하기 어렵고, 테스트와 오류 처리가 복잡하다.

스타일	강점	약점
마이크로서비스	확장성, 민첩성	복잡도, 비용
레이어드	단순함, 낮은 비용	확장성, 민첩성
이벤트 기반	확장성, 응답성	복잡도, 테스트 용이성

아키텍처 스타일은 시스템의 전반적인 모습과 특성을 정의하기 때문에 **처음부터 올바르게 선택하는 것이 중요하다.** 1층짜리 집을 3층짜리 주택으로 다시 지으려면 매우 큰 공사가 되는 것과 같다.

1.6 아키텍처와 설계

> 아키텍처는 외관보다 **구조**에 중점을 두고, 설계는 **외관**에 더 중점을 둔다.

집으로 예를 들면 다음과 같다.

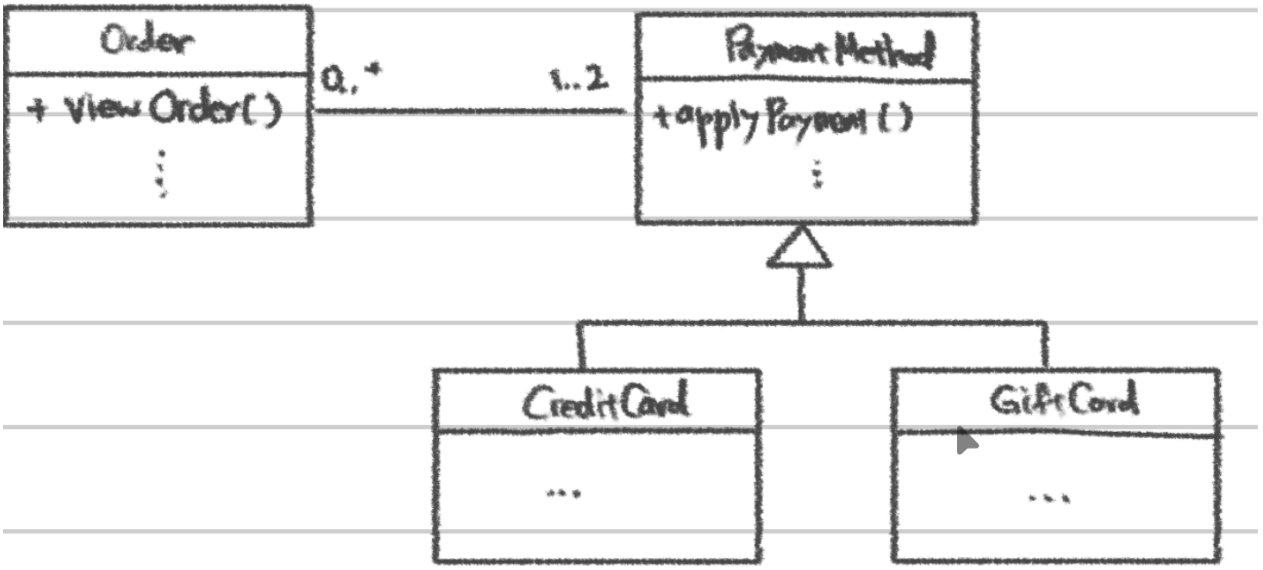
- **설계** : 벽면의 색상, 가구의 배치, 마루의 종류
- **아키텍처** : 방의 물리적 크기, 문과 창문의 배치

소프트웨어로 옮기면, 아키텍처(구조)는 웹 페이지가 서비스 및 DB와 통신해서 데이터를 조회하고 저장하는 방식과 관련된 모든 것이다. 설계는 각 페이지의 외관에 관한 내용이다. 버튼 색을 바꾸는 건 하루면 끝나지만,

화면이 DB에 직접 붙어 있던 구조를 서비스를 거치도록 바꾸는 건 훨씬 큰 작업이다.

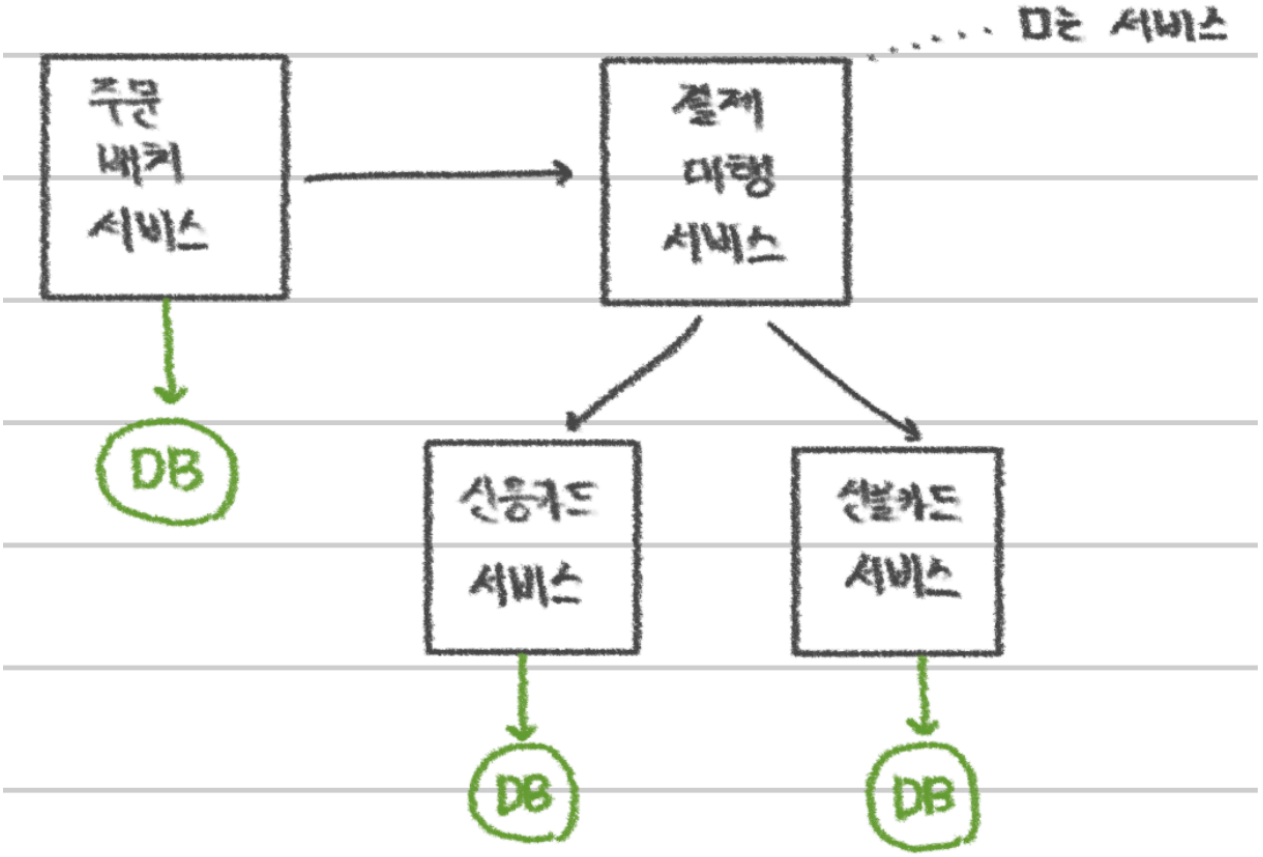
1.7 설계 관점

설계 관점에서는 기능을 구현하기 위해 **UML(Unified Modeling Language) 클래스 다이어그램**으로 클래스들이 서로 어떻게 상호작용하는지 나타낼 수 있다. 하지만 클래스 파일들이 어떻게 조직되고 배포되는지, 즉 소스코드의 물리적인 구조는 다루지 않는다.



1.8 아키텍처 관점

아키텍처는 시스템의 구조에 관한 것으로, 서비스와 DB, 그리고 서비스들이 다른 서비스나 사용자 인터페이스와 어떻게 통신하는지를 다룬다.



같은 시스템이라도 1.7의 다이어그램은 "클래스끼리 어떻게 협력하는가"를, 1.8의 다이어그램은 "서비스와 DB가 어떻게 연결되는가"를 보여준다. 확대 배율이 다른 지도라고 생각하면 된다.

1.9 아키텍처와 설계 사이

어떤 결정은 확실히 아키텍처에 속하고, 어떤 결정은 분명히 설계에 속한다. 하지만 어떤 결정은 둘 사이 어딘가에 위치한다. 아키텍처와 설계는 둘로 딱 나뉘는 것이 아니라 **하나의 스펙트럼**이다.

중간 어디에 위치하는지 굳이 신경 써야 할까?

결정이 스펙트럼의 어디에 위치하는지 알면 **누가 그 결정을 내려야 하는지** 정하는 데 도움이 된다.

- **개발팀의 결정** : 특정 기능을 구현하는 클래스 설계
- **아키텍트의 결정** : 시스템에 적합한 아키텍처 스타일

- **함께 내려야 하는 결정** : 서비스를 쪼개거나 다시 합치는 일

1.9.1 우리의 결정은 어디에 위치할까?

전략적인가, 전술적인가

- **전략적인 결정** : 장기적이며, 이후의 행동이나 결정에 영향을 미친다.
- **전술적인 결정** : 단기적이며, 일반적으로 다른 행동이나 결정과 독립적이다.

> 집의 크기는 방의 개수와 크기에 영향을 주지만, 조명을 고르는 것은 식탁의 크기에 영향을 주지 않는다. 결정이 전략적인 성향이 강할수록 아키텍처에 가깝다.

전략적인지 전술적인지는 다음 질문으로 가늠해볼 수 있다.

결정하는 데 얼마나 많은 생각과 계획이 필요한가?

- 몇 분에서 한 시간이면 결정할 수 있다면 일반적으로 전술적이다.
- 반면 며칠에서 몇 주에 걸쳐 계획해야 한다면 전략적일 가능성이 높다.

얼마나 많은 사람이 결정에 참여하는가?

- 더 많은 사람이 참여할수록 그 결정은 전략적이다.
- 혼자 결정할 수 있거나 동료 몇 명과 정할 수 있다면 전술적일 가능성이 높다.

구축하거나 변경하는 데 얼마나 많은 노력이 드는가?

- 아키텍처 결정은 구축하거나 변경하는 데 많은 노력이 든다.
- 반면 설계는 상대적으로 적은 노력이 필요하다.

1.10 트레이드오프

> 트레이드오프는 어떤 결정을 내릴 때 장단점을 비교하고 평가하는 과정이다.

중대한 트레이드오프가 포함된 결정은 많은 시간과 분석이 필요하며, 일반적으로 아키텍처에 가깝다. 반면 상대적으로 덜 중요한 결정은 빠르게 내릴 수 있고 분석도 적게 필요하므로 설계 쪽에 가깝다.

예를 들어 모놀리식에서 마이크로서비스로 갈지 고민한다면, 확장성과 민첩성을 얻는 대신 운영 복잡도와 비용을 감수해야 한다. 어느 쪽을 골라도 무언가를 잃는다. 반면 변수 이름을 무엇으로 지을지는 잃을 것이 거의 없다.

소프트웨어 아키텍처에서 **"무조건 좋은 선택"은 없다.** 모든 선택에는 대가가 따르며, 아키텍트의 역할은 정답을 찾는 것이 아니라 현재 상황에서 **가장 덜 나쁜 트레이드오프**를 고르는 것이다.

정리

구분	아키텍처	설계
관심사	구조	외관, 구현 세부사항
성격	전략적	전술적
변경 비용	크다	작다
참여 인원	많다	적다
트레이드오프	중대하다	가볍다
집으로 비유	방 개수, 층수, 창문 위치	벽지 색, 가구 배치, 바닥재